Networking

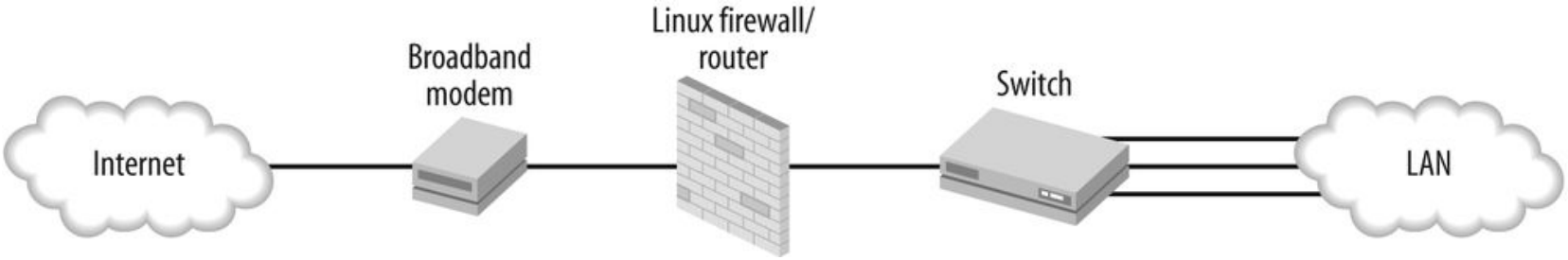
Topics:
OSI Layers
Network layer
- TCP vs UDP
- Port numbers
- Binding ports

System Layer
- Sockets(Application Programming Interfaces (API))

Application layer protocols
- HTTP (get, post request)
- WebSockets

https://www.csd.uoc.gr/~hy556/material/tutorials/cs556-3rd-tutorial.pdf

https://intronetworks.cs.luc.edu/



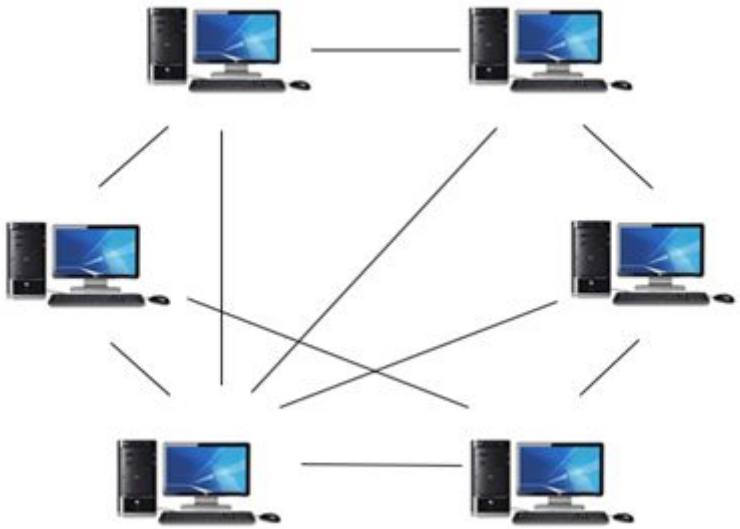
https://www.oreilly.com/library/view/linux-networking-cookbook/9780596102487/ch01.html

Linux Networking Commands

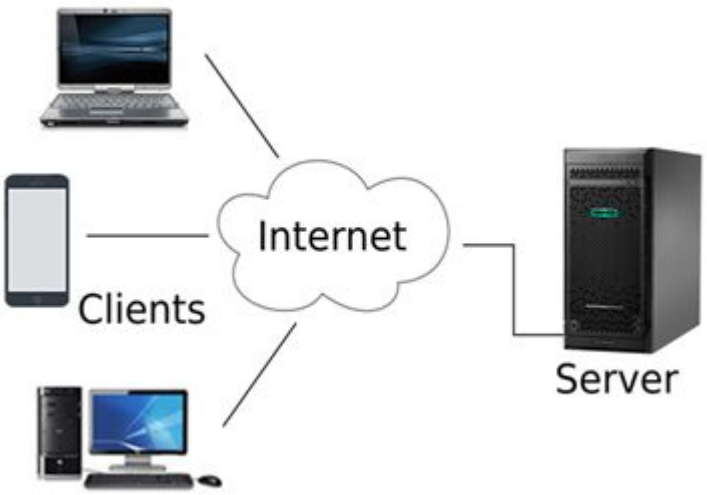
Basic Linux Network Commands

Top Linux Network Commands in 2024

Peer to peer network



Client Server network

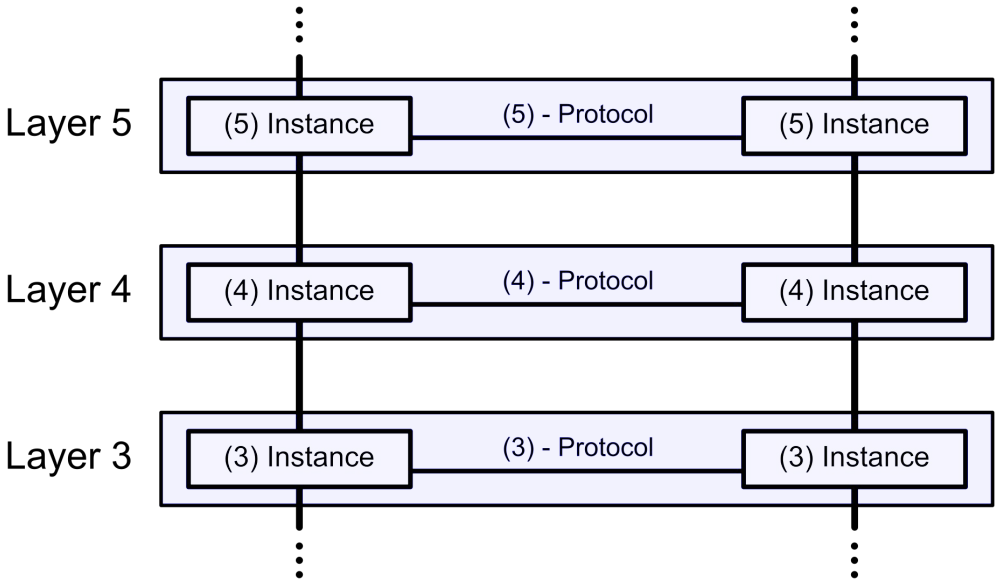


https://www.javatpoint.com/types-of-computer-network

Communication

Conceptual Layering of Network Architecture

Successful communication requires **a protocol(an agreement mechanism)** on every layer of the systems



https://en.wikipedia.org/wiki/OSI_model

OSI model

by layer

7. Application layer [hide]

NNTP · SIP · SSI · DNS · FTP · Gopher · HTTP · NFS · NTP · SMPP · SMTP · SNMP · Telnet · DHCP · NETCONF · *more....*

6. Presentation layer [hide]

MIME · XDR · ASN.1 · ASCII · PGP

5. Session layer [hide]

Named pipe · NetBIOS · SAP · PPTP · RTP · SOCKS · X.225[1]

4. Transport layer [hide]

TCP · UDP · SCTP · DCCP · SPX

3. Network layer [hide]

IP (IPv4 · IPv6) · ICMP · IPsec · IGMP · IPX · IS-IS · AppleTalk · X.25 · PLP

2. Data link layer [hide]

ATM · ARP · SDLC · HDLC · CSLIP · SLIP · GFP · PLIP · IEEE 802.2 · LLC · MAC · L2TP · IEEE 802.3 · Frame Relay · ITU-T G.hn DLL · PPP · X.25 LAPB · Q.922 LAPF

1. Physical layer [hide]

RS-232 · RS-449 · ITU-T V-Series · I.430 · I.431 · PDH · SONET/SDH · PON · OTN · DSL · IEEE 802.3 · IEEE 802.11 · IEEE 802.15 · IEEE 802.16 · IEEE 1394 · ITU-T G.hn PHY · USB · Bluetooth

V · T · E

Conceptual model (NOT USED in Practice)

Internet protocol suite

Application layer

BGP · DHCP (v6) · DNS · FTP · HTTP (HTTP/3) · HTTPS · IMAP · IRC · LDAP · MGCP · MQTT · NNTP · NTP · OSPF · POP · PTP · ONC/RPC · RTP · RTSP · RIP · SIP · SMTP · SNMP · SSH · Telnet · TLS/SSL · XMPP · *more...*

Transport layer

TCP · UDP · DCCP · SCTP · RSVP · QUIC · *more...*

Internet layer

IP (v4 · v6) · ICMP (v6) · NDP · ECN · IGMP · IPsec · *more...*

Link layer

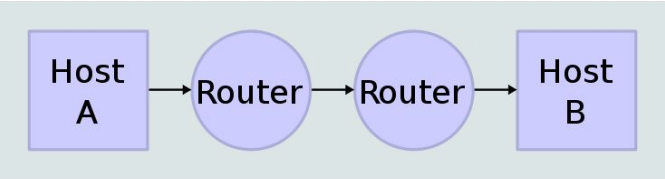
Tunnels · PPP · MAC · *more...*

V · T · E

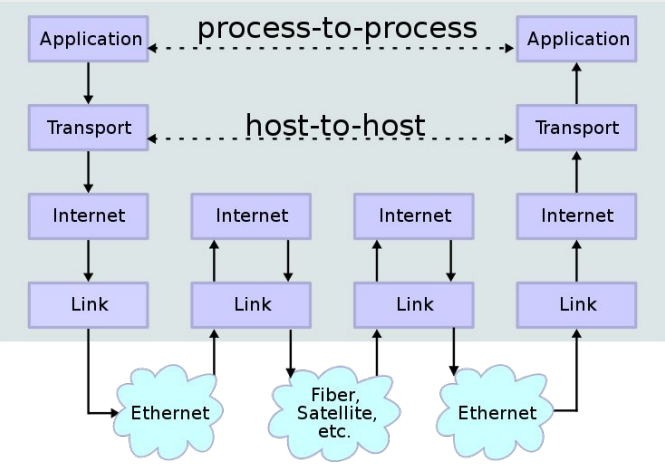
https://en.wikipedia.org/wiki/OSI_model
https://en.wikipedia.org/wiki/Internet_protocol_suite

Internet protocol suite (TCP/IP model)

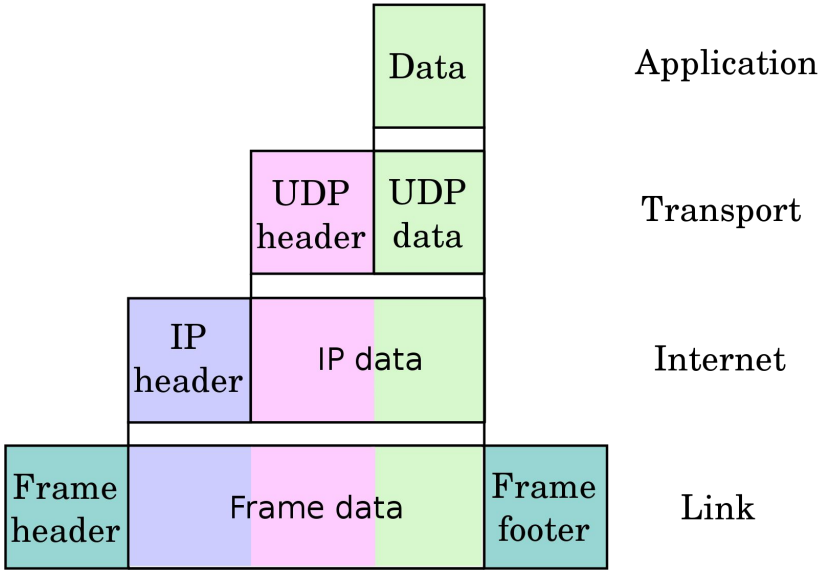
Network Topology



Data Flow



An example packet (the data that is transferred)



https://en.wikipedia.org/wiki/Internet_protocol_suite

Internet Protocol (IP)

Internet protocol suite

Application layer

BGP · DHCP (v6) · DNS · FTP · HTTP (HTTP/3) · HTTPS · IMAP · IRC · LDAP · MGCP · MQTT · NNTP · NTP · OSPF · POP · PTP · ONC/RPC · RTP · RTSP · RIP · SIP · SMTP · SNMP · SSH · Telnet · TLS/SSL · XMPP · *more...*

Transport layer

TCP · UDP · DCCP · SCTP · RSVP · QUIC · *more...*

Internet layer

IP (v4 · v6) · ICMP (v6) · NDP · ECN · IGMP · IPsec · *more...*

Link layer

Tunnels · PPP · MAC · *more...*

V · T · E

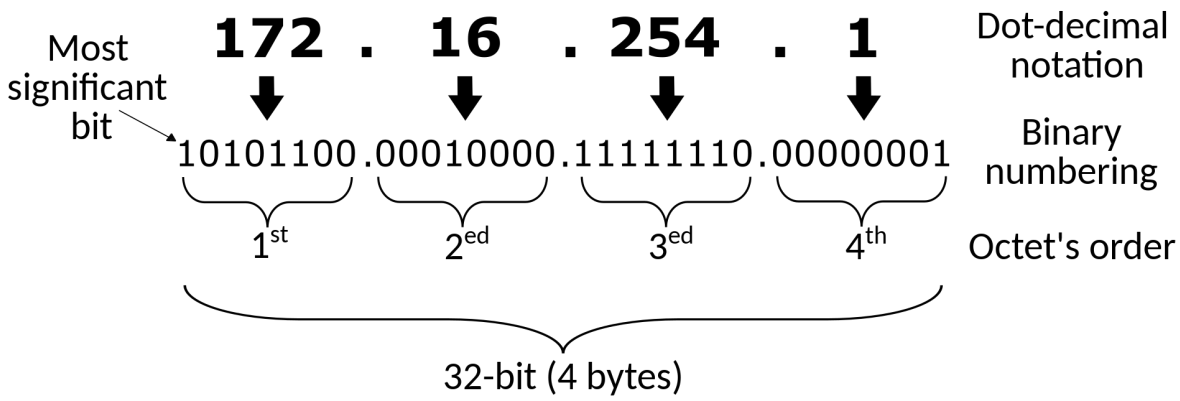
IP has the task of delivering packets from the source host to the destination host solely based on the IP addresses in the packet headers.

- Packets must contain an IP address of its destination
- Packets are handled and delivered independently
- Packets may lost, reordered or duplicated

https://en.wikipedia.org/wiki/Internet_protocol_suite

Internet Protocol (IP)

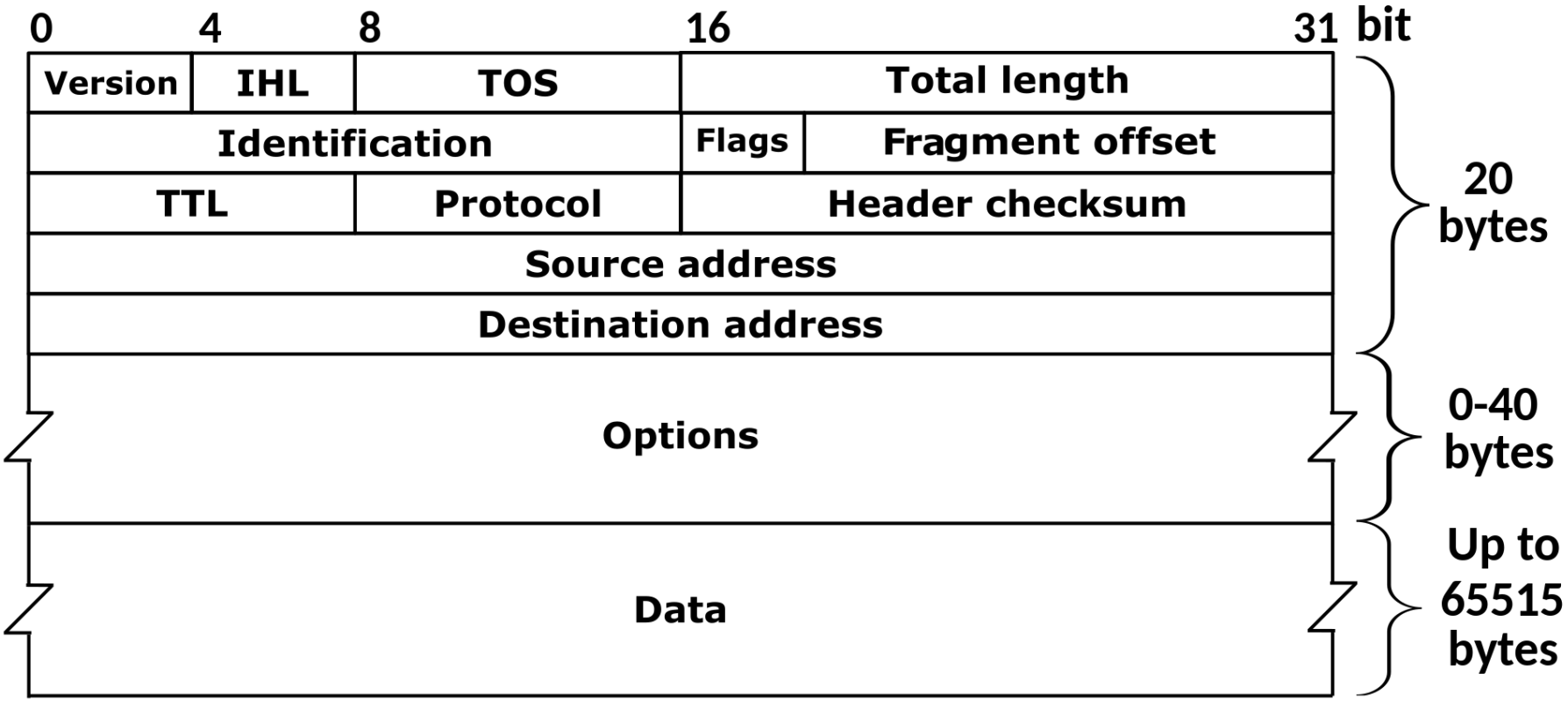
IPv4



- IPv4 – `192.168.10.150`
- IPv6 – `3002:0bd6:0000:0000:0000:ee00:0033:6778`

https://en.wikipedia.org/wiki/Internet_Protocol_version_4

IPv4 packet



https://en.wikipedia.org/wiki/Internet_Protocol_version_4

IPv6 packet header

Fixed header format

Offsets	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	*Version*				*Traffic class*								*Flow label*																			
4	32	*Payload length*																*Next header*								*Hop limit*							
8	64	*Source address*																															
12	96																																
16	128																																
20	160																																
24	192	*Destination address*																															
28	224																																
32	256																																
36	288																																

https://en.wikipedia.org/wiki/IPv6

TCP vs UDP

https://en.wikipedia.org/wiki/Internet_protocol_suite

Both use **port numbers**

- application-specific construct serving as a communication endpoint
- 16-bit unsigned integer, thus ranging from 0 to 65535) to provide end-to-end transport

```
$ lsof -i -P -n
COMMAND  PID     USER   FD   TYPE DEVICE SIZE/OFF NODE NAME
firefox 6636 ad_public  177u  IPv4 170052      0t0  TCP 10.5.1.74:35684->34.120.208.123:443 (ESTABLISHED)
firefox 6636 ad_public  189u  IPv4 170051      0t0  TCP 10.5.1.74:34326->34.149.100.209:443 (ESTABLISHED)
firefox 6636 ad_public  194u  IPv4 158123      0t0  TCP 10.5.1.74:51238->172.66.41.27:443 (ESTABLISHED)
chrome  8471 ad_public  234u  IPv4  33288      0t0  UDP 224.0.0.251:5353
chrome  8518 ad_public   35u  IPv4   3910      0t0  TCP 10.5.1.74:45734->95.183.223.253:443 (ESTABLISHED)
$ sudo netstat -tulpn
Proto Recv-Q Send-Q Local Address           Foreign Address         State       PID/Program name
tcp        0      0 127.0.0.1:3306          0.0.0.0:*               LISTEN      1531/mysqld
```

TCP vs UDP

Internet protocol suite

Application layer

BGP · DHCP (v6) · DNS · FTP · HTTP (HTTP/3) · HTTPS · IMAP · IRC · LDAP · MGCP · MQTT · NNTP · NTP · OSPF · POP · PTP · ONC/RPC · RTP · RTSP · RIP · SIP · SMTP · SNMP · SSH · Telnet · TLS/SSL · XMPP · *more...*

Transport layer

TCP · UDP · DCCP · SCTP · RSVP · QUIC · *more...*

Internet layer

IP (v4 · v6) · ICMP (v6) · NDP · ECN · IGMP · IPsec · *more...*

Link layer

Tunnels · PPP · MAC · *more...*

V · T · E

https://en.wikipedia.org/wiki/Internet_protocol_suite

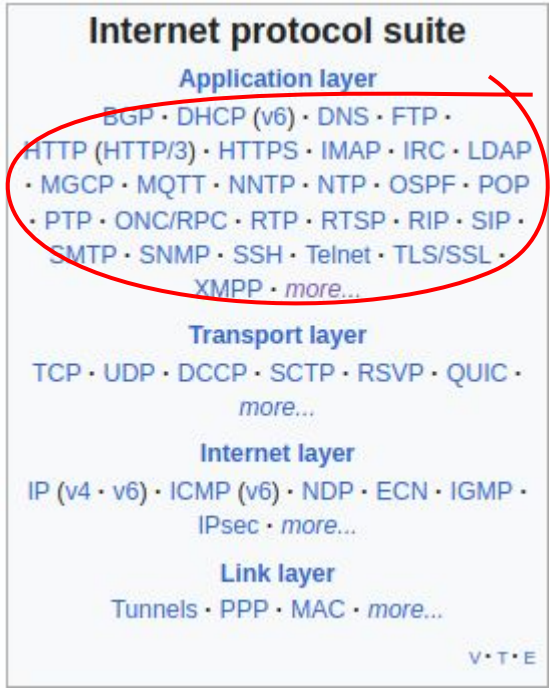
UDP: User Datagram Protocol

- no acknowledgements
- no retransmissions
- out of order, duplicates possible
- connectionless, i.e., app indicates destination for each packet

TCP: Transmission Control Protocol

- reliable byte-stream channel (in order, all arrive, no duplicates)
- similar to file I/O
- flow control
- connection-oriented
- bidirectional

Application Layer Protocols



https://en.wikipedia.org/wiki/Internet_protocol_suite

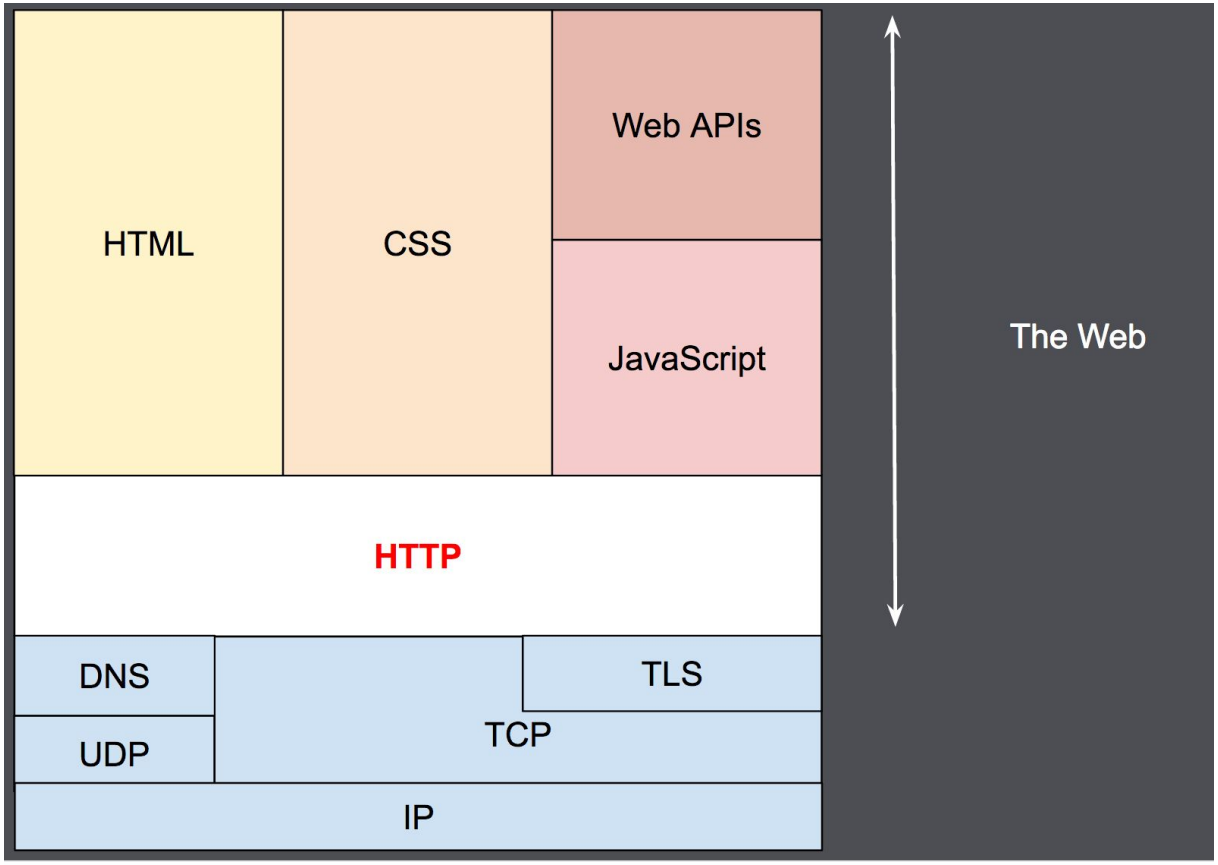
Websockets, HTTP, HTTPS

always run WebSocket, HTTP over [Transport Layer Security - Wikipedia]

HTTPs, wss (“secure” websocket)

[The WebSocket API (WebSockets) - Web APIs | MDN]

Http overview

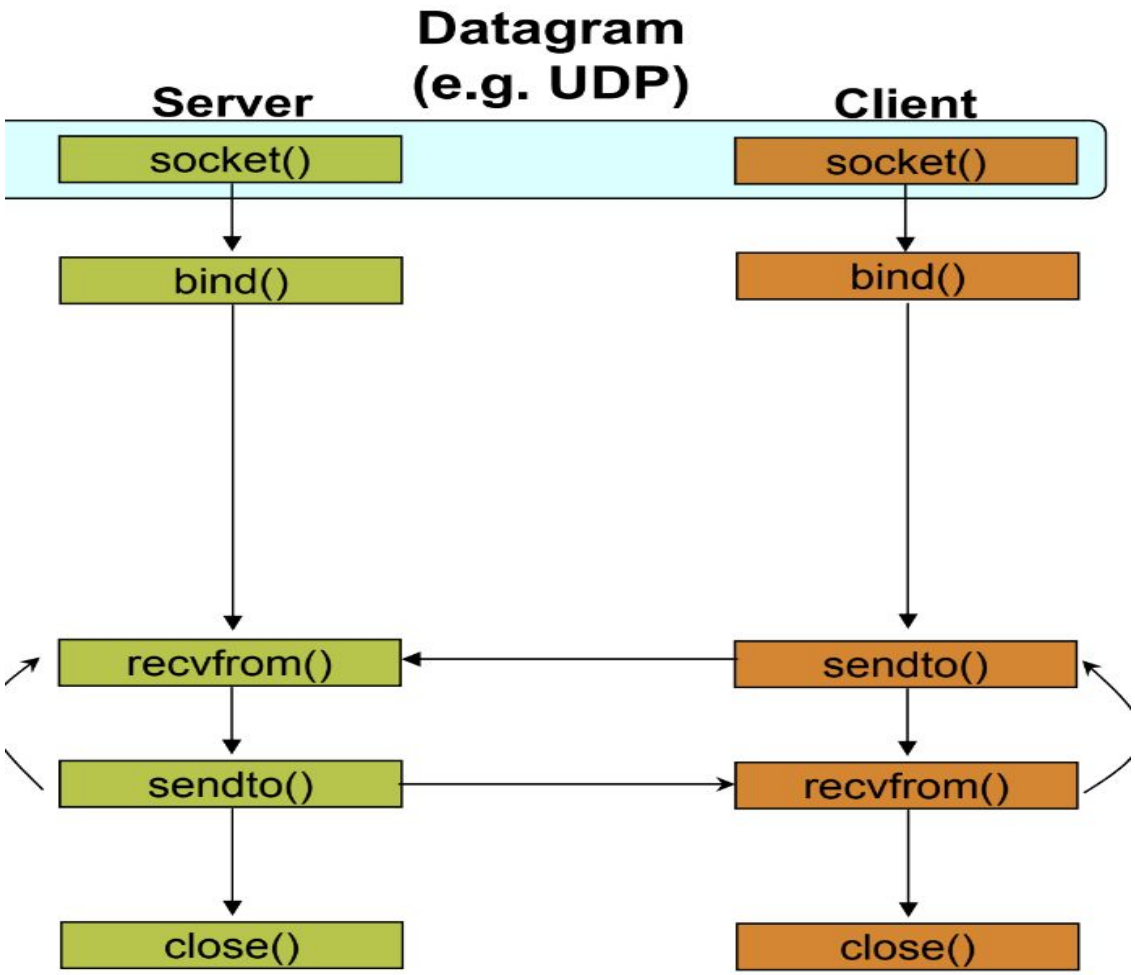


https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

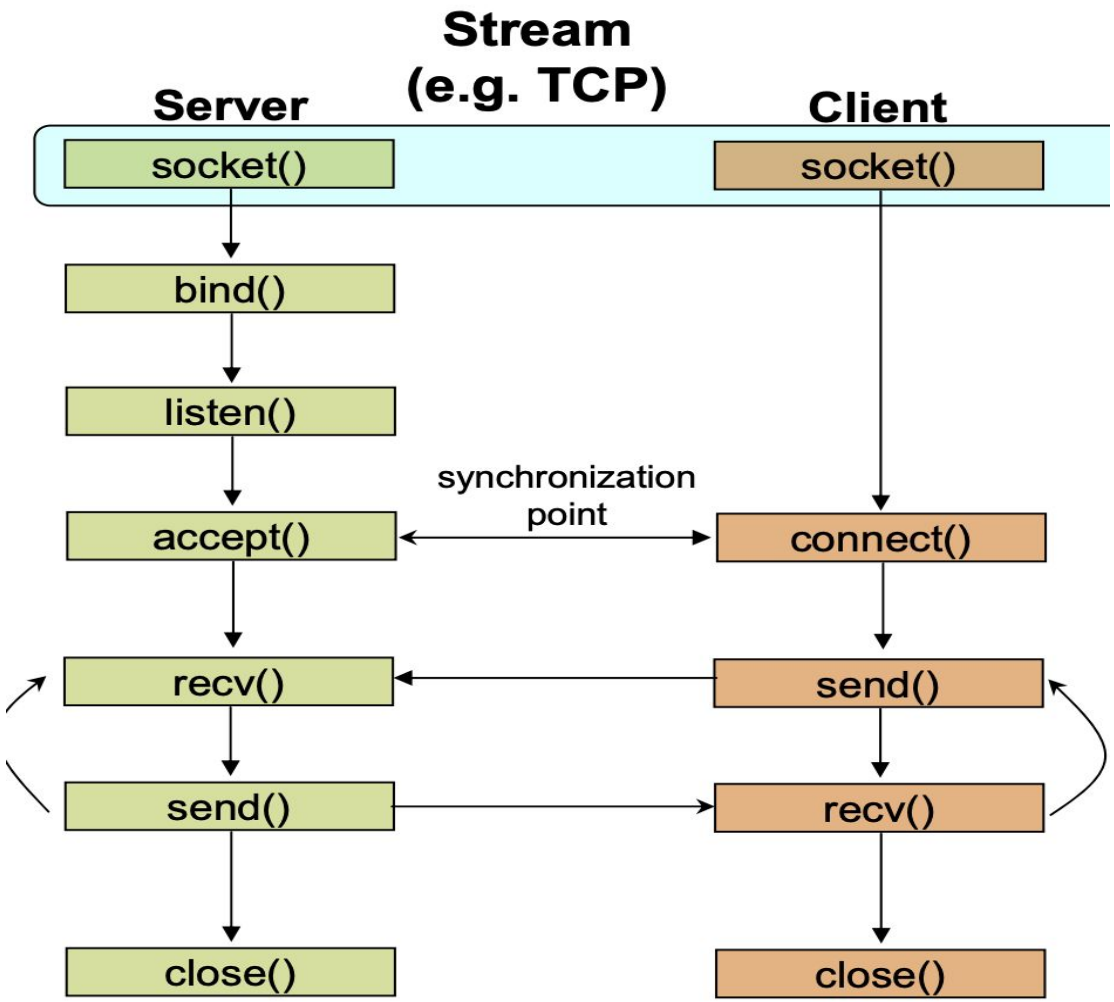
Socket Programming: System calls (API) for networking

Berkeley Sockets (System calls: API Berkeley Sockets

- Sockets(Application Programming Interfaces (API)
- Sockets are not protocol
- We will see socket programming
  - That uses UDP
  - That uses TCP
    - And see kernel implementation of Transport Layer Security(TLS) encryption protocol over TCP
    - OpenSSL
    - https://www.kernel.org/doc/html/latest/networking/tls.html



https://www.csd.uoc.gr/~hy556/material/tutorials/cs556-3rd-tutorial.pdf

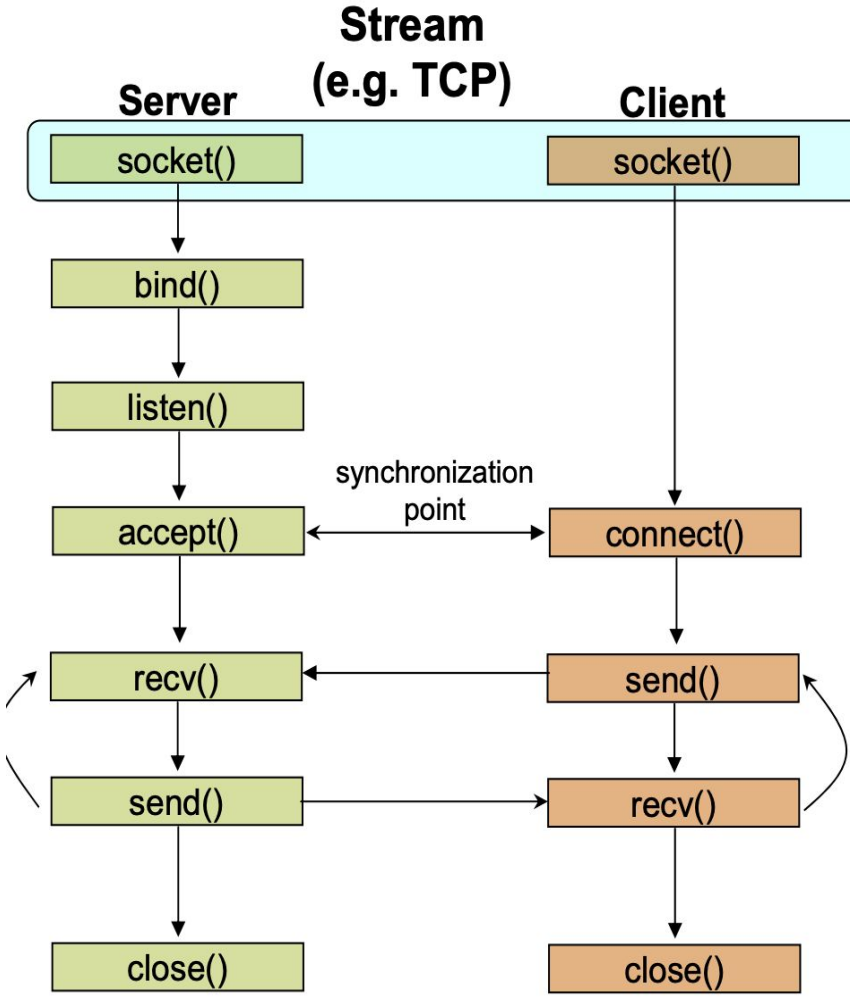


https://www.csd.uoc.gr/~hy556/material/tutorials/cs556-3rd-tutorial.pdf

socket()

creates an endpoint for communication and returns a file descriptor that refers to that endpoint.

```
#include <sys/socket.h>
int socket(int domain, int type, int protocol);
```



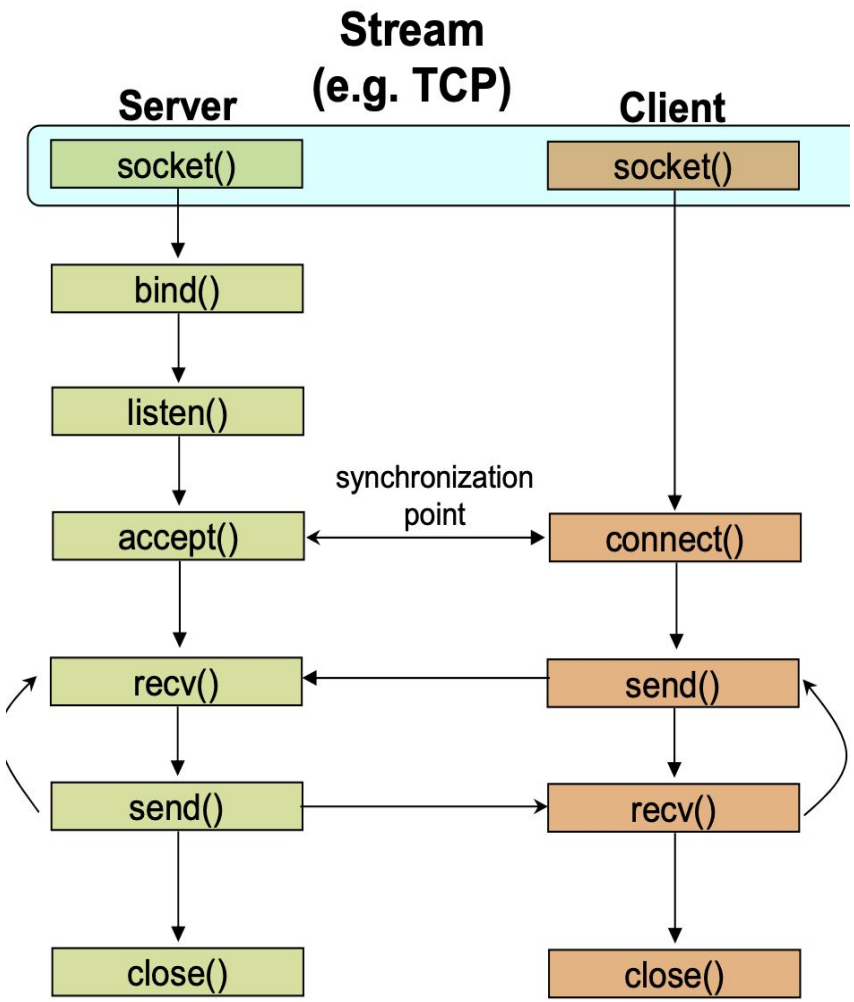
[Berkeley sockets - Wikipedia]

[https://www.csd.uoc.gr/~hy556/material/tutorials/cs556-3rd-tutorial.pdf]

socket()

creates an endpoint for communication and returns a file descriptor that refers to that endpoint.

```
#include <sys/socket.h> /*for socket*/
#include <netinet/in.h> /*for PF_INET, AF_INET,
IPPPROTO_TCP and etc. */
int main(){
   int sock = socket(AF_INET, SOCK_STREAM,
                     IPPROTO_TCP);

}
```



Socket structures ($ man sockaddr)

```
#include <sys/socket.h>

struct sockaddr {
   sa_family_t     sa_family;      /* Address family */
   char            sa_data[];      /* Socket address */
};
```

Internet domain sockets

```c
#include <netinet/in.h>

struct sockaddr_in {
    sa_family_t     sin_family;     /* AF_INET */
    in_port_t       sin_port;       /* Port number */
    struct in_addr  sin_addr;       /* IPv4 address */
};
struct in_addr {
    in_addr_t s_addr; /* IPv4 address */
};
struct sockaddr_in6 {
    sa_family_t     sin6_family;    /* AF_INET6 */
    in_port_t       sin6_port;      /* Port number */
    uint32_t        sin6_flowinfo;  /* IPv6 flow info */
    struct in6_addr sin6_addr;      /* IPv6 address */
    uint32_t        sin6_scope_id;  /* Set of interfaces for a scope */
};
```

UNIX domain sockets

```
#include <sys/un.h>

struct sockaddr_un {
    sa_family_t     sun_family;     /* Address family */
    char            sun_path[];     /* Socket pathname */
};
```

Server side socket

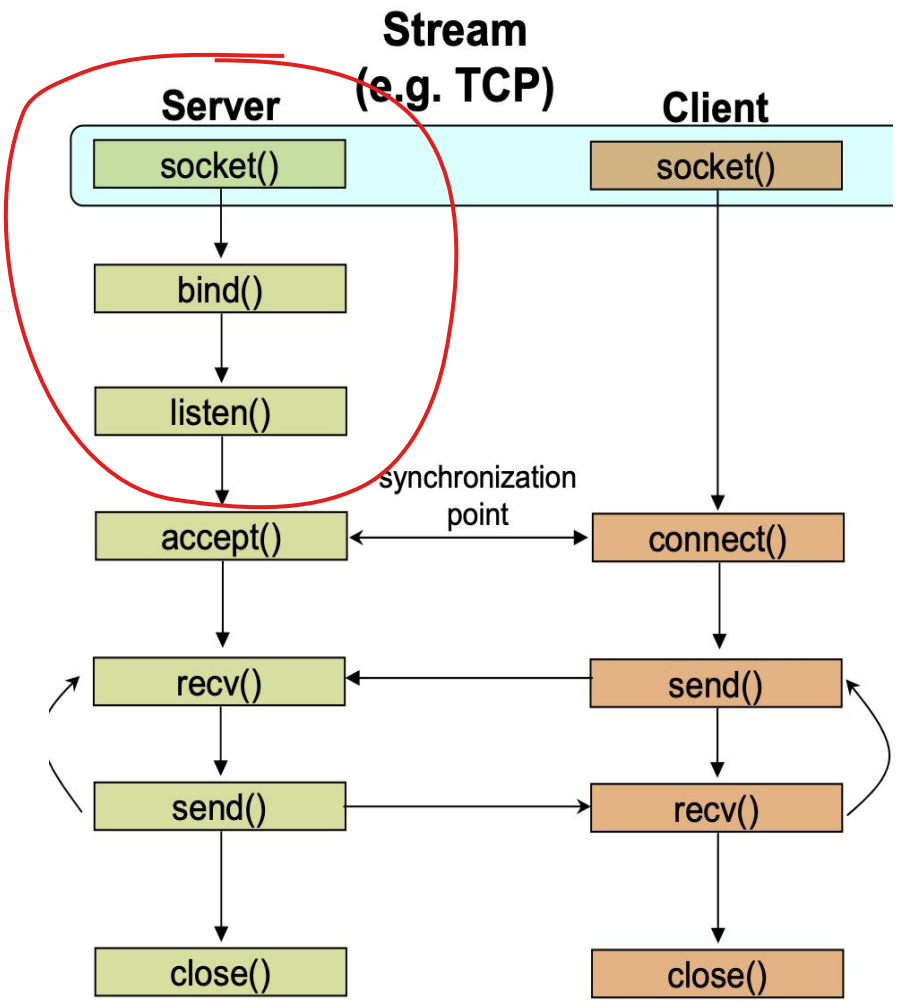
```
#include <sys/socket.h> /*for socket(int domain, int type, int protocol);*/
#include <netinet/in.h> /*for PF_INET, AF_INET, IPPPROTO_TCP and etc. */
#include <arpa/inet.h> /*htonl, htons, ntohl, ntohs: convert values between host and
network byte order */
#include <string.h>
#define PORT_NUMBER 5100
int main(){

   struct sockaddr_in my_addr;
   my_addr.sin_addr.s_addr = htonl(INADDR_ANY);
   my_addr.sin_family = AF_INET;
   my_addr.sin_port = htons(PORT_NUMBER);
```

Client side socket

```
#include <sys/socket.h> /*for socket(int domain, int type, int protocol);*/
#include <netinet/in.h> /*for PF_INET, AF_INET, IPPPROTO_TCP and etc. */
#include <arpa/inet.h>  /*for htonl, htons, ntohl, ntohs: convert values between host and
network byte order */
#include <string.h>     /*for memset*/

#define PORT_NUMBER 5100
#define SERVER_IP_ADDRESS "127.0.0.1"
int main(){
   struct sockaddr_in srv_addr;
   memset(&srv_addr, 0, sizeof(srv_addr));
   srv_addr.sin_addr.s_addr = inet_addr(SERVER_IP_ADDRESS);
   srv_addr.sin_family = AF_INET;
   srv_addr.sin_port = htons (PORT_NUMBER);
```

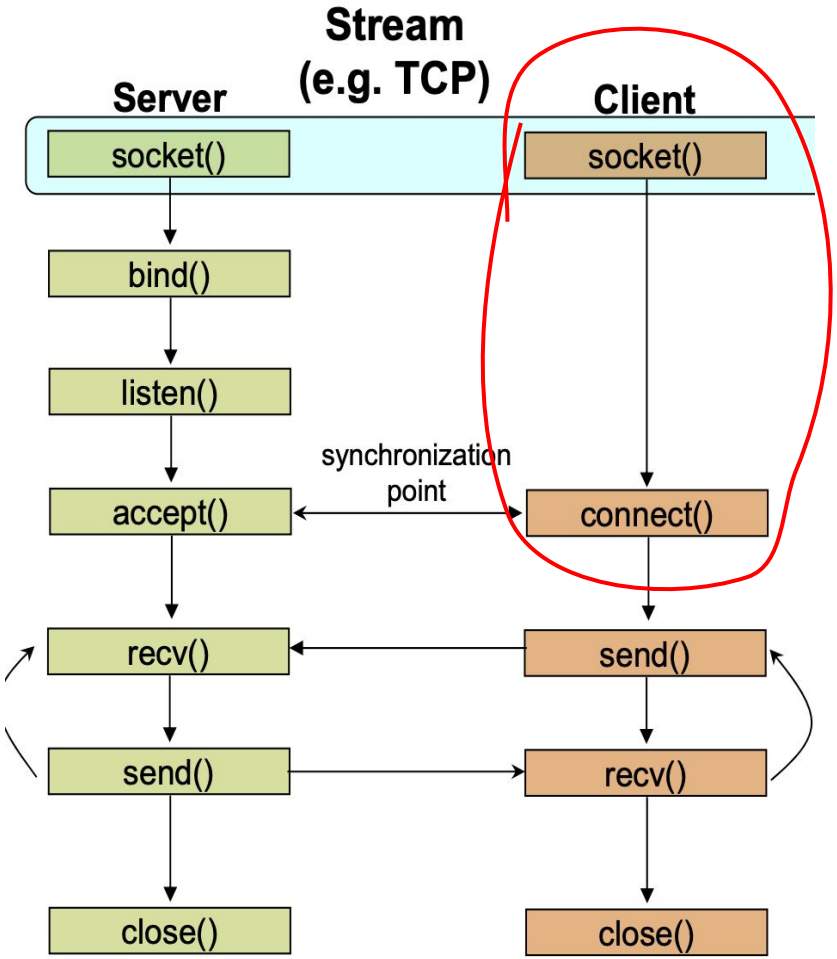
Server side

```
#define NUMBER_OF_CONNECTIONS 10
#define PORT_NUMBER 5100
int main(){

    struct sockaddr_in my_addr;
    my_addr.sin_addr.s_addr = htonl(INADDR_ANY);
    my_addr.sin_family = AF_INET;
    my_addr.sin_port = htons(PORT_NUMBER);

    int sfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if(sfd == -1){
        perror("socket");
        exit(1);
    }
    if(bind(sfd, &my_addr, sizeof(my_addr)) == -1){
        perror("bind");
        exit(1);
    }
    if(listen(sfd, NUMBER_OF_CONNECTIONS) == -1){
        perror("listen");
        exit(1);
    }
```



client side

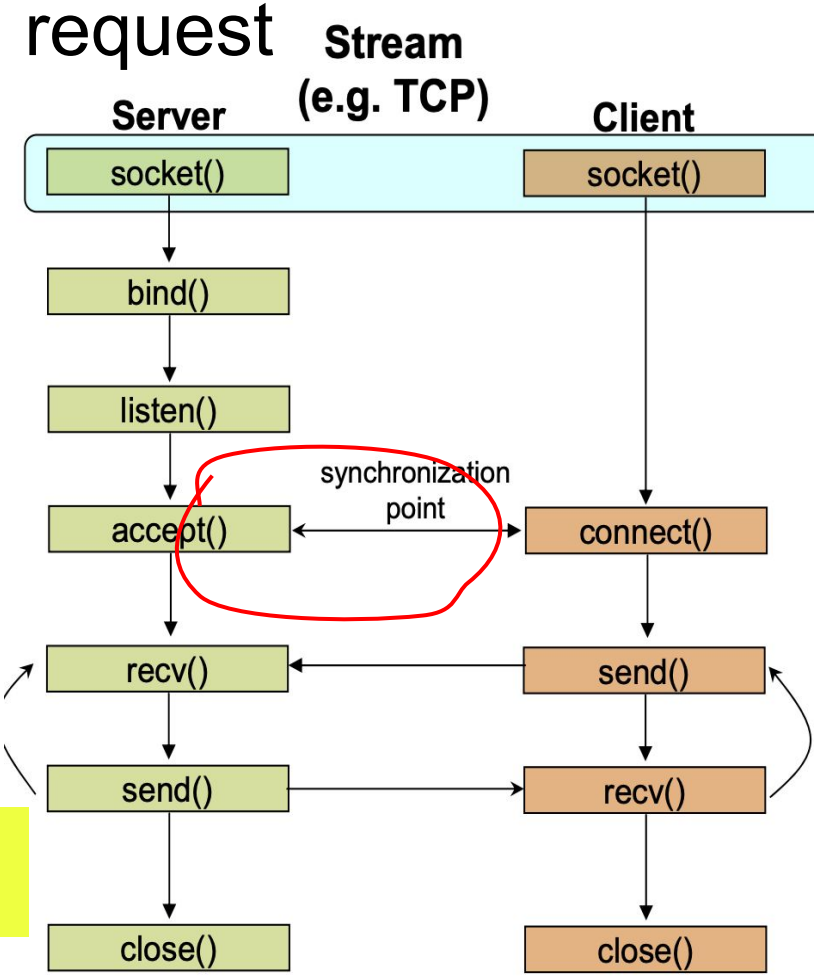
```c
#define PORT_NUMBER 5100
#define SERVER_IP_ADDRESS "127.0.0.1"
int main(){
    struct sockaddr_in srv_addr;
    memset(&srv_addr, 0, sizeof(srv_addr));
    int port_number = PORT_NUMBER; /*a port number*/
    srv_addr.sin_addr.s_addr = inet_addr(SERVER_IP_ADDRESS);
    srv_addr.sin_family = AF_INET;
    srv_addr.sin_port = htons (port_number);

    int sfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
    if(sfd == -1){
        perror("socket");
        exit(1);
    }
    if(connect(sfd, &srv_addr, sizeof(srv_addr)) == -1){
        perror("connect");
        exit(1);
    }
```



https://en.wikipedia.org/wiki/Berkeley_sockets

Server side-accept a connection request

```
struct sockaddr_in accepted_addr;
memset(&accepted_addr, 0, sizeof(accepted_addr));
socklen_t size = sizeof(accepted_addr);
int sfd_accepted = accept(sfd, (struct sockaddr *)
&accepted_addr, &size);
if( sfd_accepted == -1){
   perror("accept");
   exit(1);
}
```



`sfd_accepted:` We have new socket for the accepted connection. This does not listen and affect the original socket.

https://en.wikipedia.org/wiki/Berkeley_sockets

Send message on a socket

```
#include <sys/socket.h>
/*With a  zero  flags  argument, send() is equivalent to write(2)*/

ssize_t send(int sockfd, const void buf[.len], size_t len, int flags);
ssize_t sendto(int sockfd, const void buf[.len], size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
ssize_t sendmsg(int sockfd, const struct msghdr *msg, int flags);
```

```
send(sockfd, buf, len, flags);
      is equivalent to
sendto(sockfd, buf, len, flags, NULL, 0);
```

Receive message from a socket

```
#include <sys/socket.h>
/*With a zero flags argument, recv() is generally equivalent to read(2)*/

ssize_t recv(int sockfd, void buf[.len], size_t len,
             int flags);
ssize_t recvfrom(int sockfd, void buf[restrict .len], size_t len,
             int flags,
             struct sockaddr *_Nullable restrict src_addr,
             socklen_t *_Nullable restrict addrlen);
ssize_t recvmsg(int sockfd, struct msghdr *msg, int flags);
```

```
recv(sockfd, buf, len, flags);
      is equivalent to
recvfrom(sockfd, buf, len, flags, NULL, NULL);
```

Client send and recv message

```
/*send recv message*/
   char msg[] = "client says hi";
   char buf[256] = {0};
   int b = send(sfd, msg, sizeof(msg), 0);
   printf("client sent %d byte msg\n", b);

   int r = recv(sfd, buf, sizeof(buf), 0);
   if (r != -1){
       printf("client: incoming message: %.256s", buf);
   }
```

Server: recv and send message

```
char buf[256] = {0};

/*recv send message*/
int r = recv(sfd_accepted, &buf[0], sizeof(buf), 0);

if (r != -1){
    printf("server: incoming message: %.256s", buf);
    char msg[] = "server says hi";
    int b = send(sfd_accepted, msg, sizeof(msg), 0);
    printf("server sent %d byte msg\n", b);
}
```

Full server.c

```c
#include <sys/socket.h> /*for socket(int domain, int type, int protocol);*/
#include <netinet/in.h> /*for PF_INET, AF_INET, IPPPROTO_TCP and etc. */
#include <arpa/inet.h> /*htonl, htons, ntohl, ntohs: convert values between host
and network byte order */
#include <string.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#define NUMBER_OF_CONNECTIONS 10
#define PORT_NUMBER 5100
int main(){
   struct sockaddr_in my_addr;
   my_addr.sin_addr.s_addr = htonl(INADDR_ANY);
   my_addr.sin_family = AF_INET;
   my_addr.sin_port = htons(PORT_NUMBER);

   int sfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
   if(sfd == -1){
       perror("socket");
       exit(1);
   }
   if(bind(sfd, (struct sockaddr *) &my_addr, sizeof(my_addr)) == -1){
       perror("bind");
       exit(1);
   }
   if(listen(sfd, NUMBER_OF_CONNECTIONS) == -1){
       perror("listen");
       exit(1);
   }
```

```c
struct sockaddr_in accepted_addr;
memset(&accepted_addr, 0, sizeof(accepted_addr));
socklen_t size = sizeof(accepted_addr);
int sfd_accepted = accept(sfd,  (struct sockaddr *) &accepted_addr, &size);
if( sfd_accepted == -1){
    perror("accept");
    exit(1);
}
char buf[256] = {0};

/*recv send message*/
int r = recv(sfd_accepted, &buf[0], sizeof(buf), 0);
if (r != -1){
    printf("server: incoming message: %.256s\n", buf);
    char msg[] = "server says hi";
    int b = send(sfd_accepted, msg, sizeof(msg), 0);
    printf("server sent %d byte msg\n", b);
}

close(sfd_accepted);
close(sfd);
```

Full client.c

```c
#include <sys/socket.h> /*for socket(int domain, int type,
int protocol);*/
#include <netinet/in.h> /*for PF_INET, AF_INET, IPPPROTO_TCP
and etc. */
#include <arpa/inet.h>  /*for htonl, htons, ntohl, ntohs:
convert values between host and network byte order */
#include <string.h>     /*for memset*/
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

#define PORT_NUMBER 5100
#define SERVER_IP_ADDRESS "127.0.0.1"
int main(){
   struct sockaddr_in srv_addr;
   memset(&srv_addr, 0, sizeof(srv_addr));
   srv_addr.sin_addr.s_addr = inet_addr(SERVER_IP_ADDRESS);
   srv_addr.sin_family = AF_INET;
   srv_addr.sin_port = htons (PORT_NUMBER);

   int sfd = socket(AF_INET, SOCK_STREAM, IPPROTO_TCP);
   if(sfd == -1){
       perror("socket");
       exit(1);
   }
```

```c
   if(connect(sfd, (struct sockaddr *) &srv_addr, sizeof(srv_addr)) == -1){
       perror("connect");
       exit(1);
   }

   /*send recv message*/
   char msg[] = "client says hi";
   char buf[256] = {0};
   int b = send(sfd, msg, sizeof(msg), 0);
   printf("client sent %d byte msg\n", b);
   int r = recv(sfd, buf, sizeof(buf), 0);
   if (r != -1){
       printf("client: incoming message: %.256s\n", buf);
   }

   close(sfd);
}
```

Multi-thread server

```
for(;;){

  sfd_accepted[i] = accept(sfd, (struct sockaddr *)&accepted_addr[i], &size);
  if (sfd_accepted[i] == -1) {
    perror("accept");
    exit(1);
  }
/* new_thread to handle send receive for a client */
```

Non-Blocking IO & Asynchronous IO with SIGIO

Making Socket Non-blocking

```
d = socket(AF_INET, SOCK_STREAM | SOCK_NONBLOCK, 0);
```

```
#include <fcntl.h>
int fcntl(int fd, int cmd, ... /* arg */ );
```

```
#include <sys/socket.h>
int getsockopt(int sockfd, int level, int optname,
             void optval[restrict *.optlen],
             socklen_t *restrict optlen);
int setsockopt(int sockfd, int level, int optname,
             const void optval[.optlen],
             socklen_t optlen);
```

`SOCK_NONBLOCK:` `read()` will return immediately with whatever bytes are available

select(2) - Linux manual page, poll(2) - Linux manual page

- Both waits for an **array of fds** until one or more becomes ready for I/O
- You can set timeout for waits

Asynchronous IO with signals

In Lab 13, we used SIGIO for asynchronous input from terminal and pipes(IPC):

```
fcntl(fd, F_SETOWN, getpid());
```

- This changes who gets SIGIO when a change happens on fd. Another setting is

```
fcntl(fd, F_SETFL, O_ASYNC);
```

- This enables generations of SIGIO on fd.

```
fcntl(fd, F_SETFL, O_NONBLOCK);
```

- This makes calls like **`read(fd,..)`** nonblocking.

From lab13

```c
/* aio_example.c*/
#include <fcntl.h>
#include <signal.h>
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* See notes on why
"volatile sig_atomic_t" is better*/
volatile sig_atomic_t canread = 0;

void sigio_handler(int signo) {
   canread = 1;
   return;
}
```

```c
int main() {
   /*...*/
   fd = accept(sfd, (struct sockaddr *)&accepted_addr[i], &size);
   /*...*/

   fcntl(fd, F_SETOWN, getpid());
   fcntl(fd, F_SETFL, O_ASYNC);

   /*set signal handler*/
   struct sigaction action = {0};
   action.sa_handler = &sigio_handler;

   if (sigaction(SIGIO, &action, NULL) == -1) {
       perror("signal action");
       exit(EXIT_FAILURE);
   }
   /*...if canread = 0, do sth else*/
```

For more see https://www.csd.uoc.gr/~hy556/material/tutorials/cs556-3rd-tutorial.pdf

Application layer protocols

Application Layer Protocols

Internet protocol suite

Application layer

BGP · DHCP (v6) · DNS · FTP · HTTP (HTTP/3) · HTTPS · IMAP · IRC · LDAP · MGCP · MQTT · NNTP · NTP · OSPF · POP · PTP · ONC/RPC · RTP · RTSP · RIP · SIP · SMTP · SNMP · SSH · Telnet · TLS/SSL · XMPP · *more...*

Transport layer

TCP · UDP · DCCP · SCTP · RSVP · QUIC · *more...*

Internet layer

IP (v4 · v6) · ICMP (v6) · NDP · ECN · IGMP · IPsec · *more...*

Link layer

Tunnels · PPP · MAC · *more...*

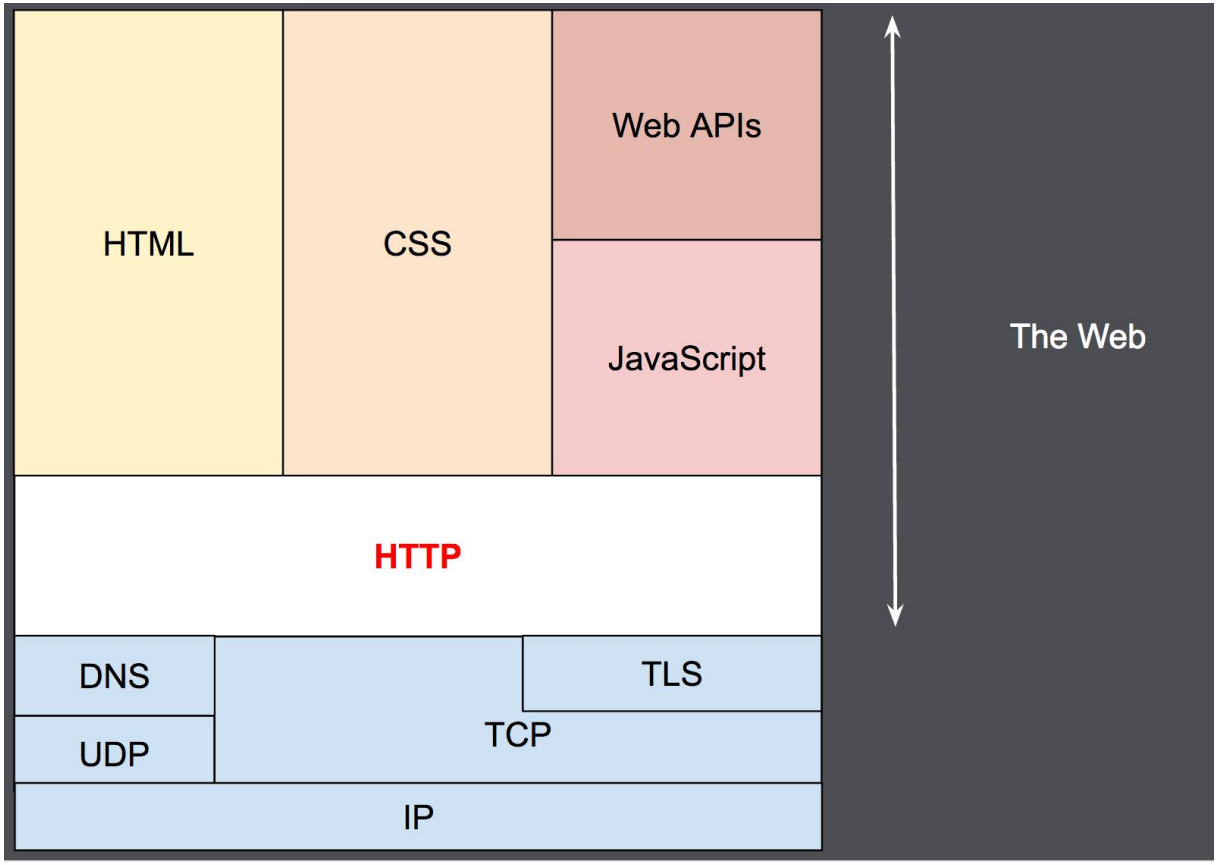
V · T · E

Websockets, HTTP, HTTPS

always run WebSocket, HTTP over [Transport Layer Security - Wikipedia]

HTTPs, wss (“secure” websocket)

Http overview



https://en.wikipedia.org/wiki/HTTP

https://developer.mozilla.org/en-US/docs/Web/HTTP/Overview

An example Http request/response

Client request

```
GET / HTTP/1.1
Host: www.example.com
User-Agent: Mozilla/5.0
Accept:
text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,*/*;q=0.8
Accept-Language: en-GB,en;q=0.5
Accept-Encoding: gzip, deflate, br
Connection: keep-alive
```

Server response

```
HTTP/1.1 200 OK
Date: Mon, 23 May 2005 22:38:34 GMT
Content-Type: text/html; charset=UTF-8
Content-Length: 155
Last-Modified: Wed, 08 Jan 2003 23:11:55 GMT
Server: Apache/1.3.3.7 (Unix) (Red-Hat/Linux)
ETag: "3f80f-1b6-3e1cb03b"
Accept-Ranges: bytes
Connection: close

<html>
  <head>
    <title>An Example Page</title>
  </head>
  <body>
    <p>Hello World, this is a very simple HTML document.</p>
  </body>
</html>
```

https://en.wikipedia.org/wiki/HTTP

Websocket vs http

WebSocket - Wikipedia

WebSocket is distinct from HTTP. Both protocols are located at layer 7 in the OSI model and depend on TCP at layer 4

Although they are different, RFC 6455 states that WebSocket "is designed to work over HTTP ports 443 and 80 as well as to support HTTP proxies and intermediaries", thus making it compatible with HTTP.

Examples

The WebSocket API (WebSockets) - Web APIs | MDN(Java server, C# server, Javascript)

https://yalantis.com/blog/how-to-build-websockets-in-go/

https://www.tutorialspoint.com/websockets/websockets_quick_guide.htm

Transferring Structured Data

XML

```
<part number="1976">
 <name>Windscreen Wiper</name>
 <description>The Windscreen wiper
   automatically removes rain
   from your windscreen, if it
   should happen to splash there.
   It has a rubber <ref part="1977">blade</ref>
   which can be ordered separately
   if you need to replace it.
 </description>
</part>
```

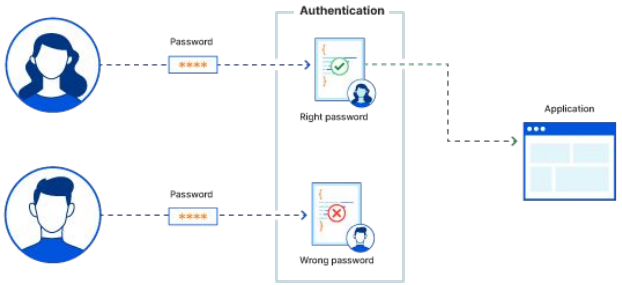
XML Essentials - W3C

JSON

```
'{"name":"John", "age":30, "car":null}'
```

https://www.w3schools.com/js/js_json_intro.asp

Access Management (authentication and authorization)



[What is OAuth? | SAML vs. OAuth | Cloudflare]

OAuth is a technical standard for authorizing users. It is a protocol for passing [authorization] from one service to another without sharing the actual user credentials, such as a username and password. With OAuth, a user can sign in on one platform and then be authorized to perform actions and view data on another platform.

[What is access control? | Authorization vs authentication | Cloudflare]

[What is authentication? | Cloudflare]

[What is SSO? | How single sign-on works | Cloudflare]

[What is SAML? | How SAML authentication works | Cloudflare]

See also OpenID [What's the difference between OpenID and OAuth? - Stack Overflow]